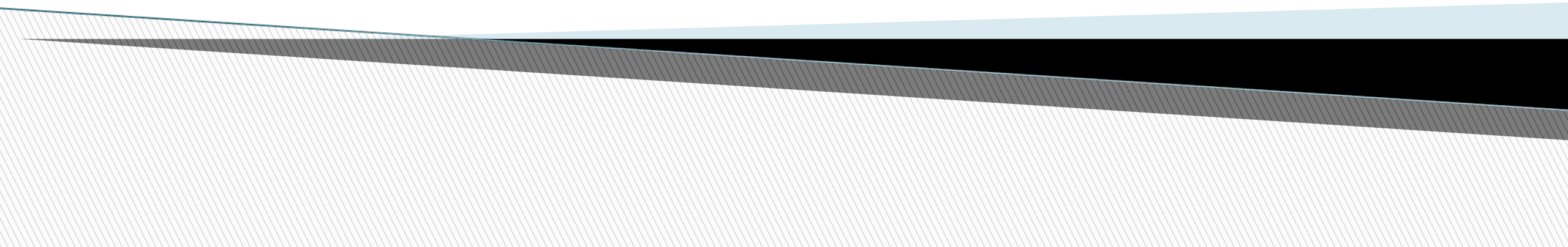
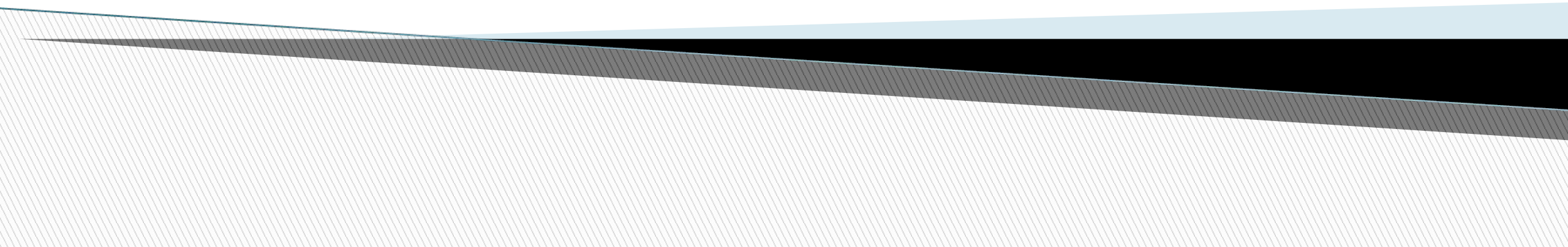


Database System

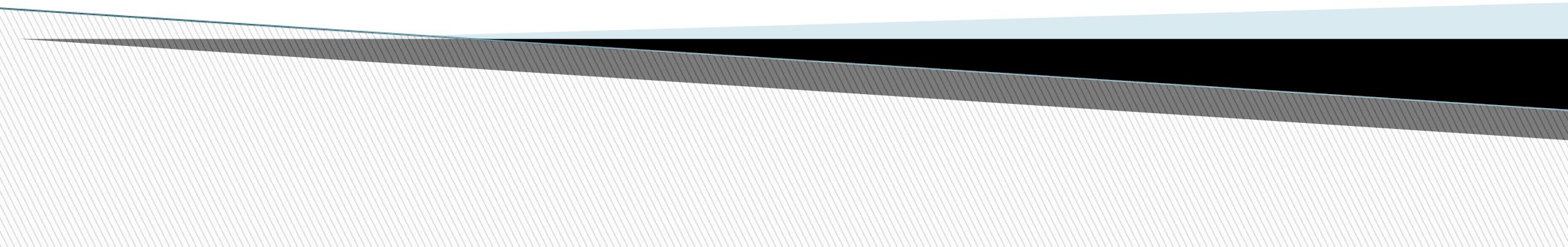
ER-Data Model



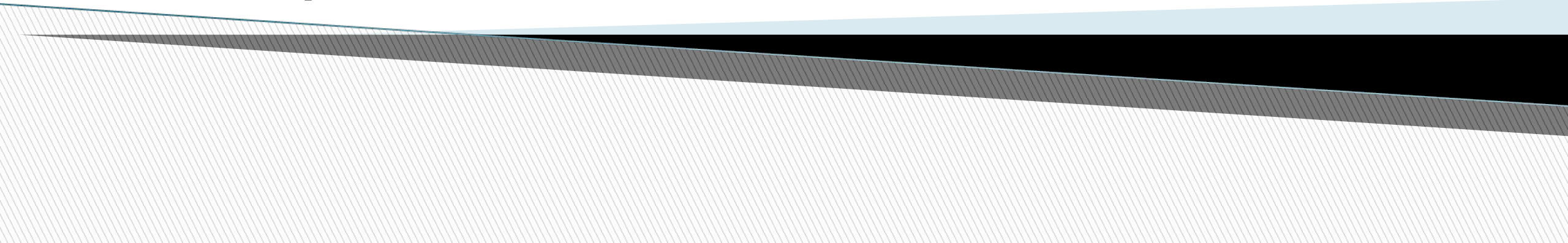
ER Data Model

- ◆ The Entity-Relationship (E-R) model is a popular conceptual data modeling technique used to represent and visualize relationships between entities in a database system.
 - ◆ The E-R model employs a graphical approach to illustrate the structure of a database, emphasizing entities, their attributes, and the relationships among them.
- 
- A decorative graphic at the bottom of the slide consisting of several overlapping, wavy horizontal bands. From top to bottom, the colors are light blue, dark blue, and a grey band with a fine diagonal line pattern.

How it differs from relational data model

- ❖ The Relational model is a practical implementation that uses tables for data storage, whereas the E-R model is a **high-level conceptual tool for visualizing entities and relationships**.
 - ❖ Relational offers the framework for database systems, combining conceptual clarity with effective implementation, whereas E-R helps with design.
- 
- The bottom of the slide features a decorative graphic consisting of several overlapping, wavy horizontal bands. From top to bottom, the colors are light blue, black, and a light gray with a fine diagonal line pattern.

Requirements Become the E-R Data Model

- After the requirements have been gathered, they are transformed into an Entity Relationship (E-R) Data Model
 - E-R Models consist of
 - Entities
 - Attributes
 - Identifiers
 - Relationships
- 
- The bottom of the slide features a decorative graphic consisting of several overlapping, wavy horizontal bands. From top to bottom, the bands are light blue, black, dark grey, and white with a fine diagonal line pattern.

E-R Diagram

- ❖ A picture showing the information created, stored, and used by a business system.
- ❖ ERD is network model that describes stored data of a system at a high level of abstraction.
- ❖ Used to design database applications
- ❖ Tool for communication between database designers and users
- ❖ Entities generally represent similar kinds of information
- ❖ Lines drawn between entities show relationships among the data
- ❖ High level business rules are also shown

Constructs in E-R Diagram

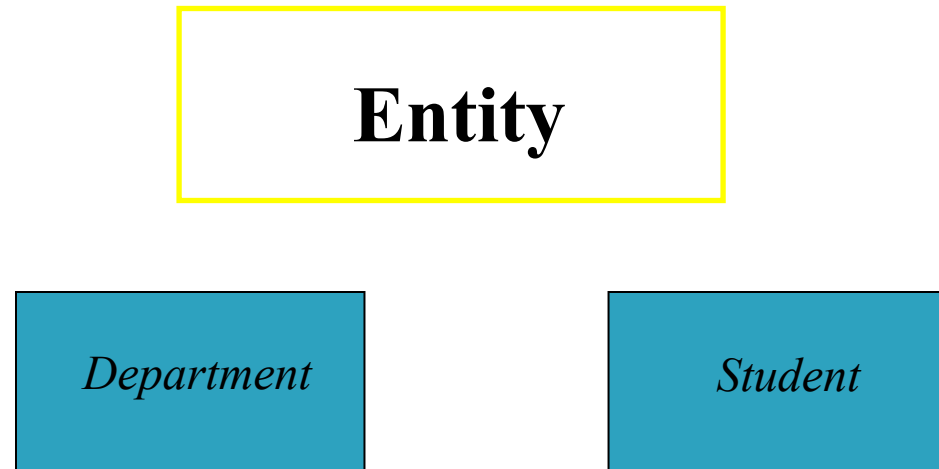
- ◆ Entity
- ◆ Attribute
- ◆ Relationship

Entity

- ❖ An entity is an object in the real world that is distinguishable from other objects
- ❖ Examples of entities:
 - *Person*: Employee, Student, Patient
 - *Place*: Store, Warehouse
 - *Object*: Machine, Product, Car
 - *Event*: Sale, Registration, Renewal
 - *Concept*: Account, Course
- ❖ Guidelines for naming and defining entity types:
 - An entity type name is a singular noun
 - An entity type should be descriptive and specific
 - An entity name should be concise

Entity Instance & Entity Type

- ❖ Entity Instance: A single occurrence of the entity (*record*)
- ❖ Entity Type: A collection of entities (*File*)



Entity Class/Entity Type vs Entity Instance

- ❖ An **entity class** is a group of entity instances that share certain characteristics (attribute values). For instance, 'Human' is an Entity Class as there are many humans.
- ❖ An **entity type** is a subgroup of entity instances that are relevant to be recognized. For instance, Child, Elderly, and Adult are entity types for the entity class "Human".

Entity Class and Entity Instance

An **entity instance** is a specific occurrence of an entity within an entity class (often corresponds to a row in a table).

An entity instance (or entity for short), is an identifiable or measurable object in the real/physical world. For instance, you are an entity instance of the human entity class.

ITEM
ItemNumber
Description
Cost
ListPrice
QuantityOnHand

Entity Class

1100
100 amp panel
\$127.50
\$170.00
14

2000
Door handle set
\$52.50
\$39.38
0

Two Entity Instances

Identify the Entities

- ◆ A college contains many departments
- ◆ Each department can offer any number of courses
- ◆ Many instructors can work in a department
- ◆ An instructor can work only in one department
- ◆ For each department there is a Head
- ◆ An instructor can be head of only one department
- ◆ Each instructor can take any number of courses
- ◆ A course can be taken by only one instructor
- ◆ A student can enroll for any number of courses
- ◆ Each course can have any number of students

From the statements given, the entities are

- Department
- Course
- Instructor
- Student

Attributes

- ❖ An entity is described using a set of attributes whose values are used to distinguish one entity from another of same type
- ❖ All entries in a given entity set have the same attributes (the values may be different).
 - E.g. name, address, weight, height are properties of a **Person** entity.
- ❖ An attribute may be required or optional
- ❖ **Domain** – the set of permitted values for each attribute
 - For example: Name can take can be text strings of a certain length

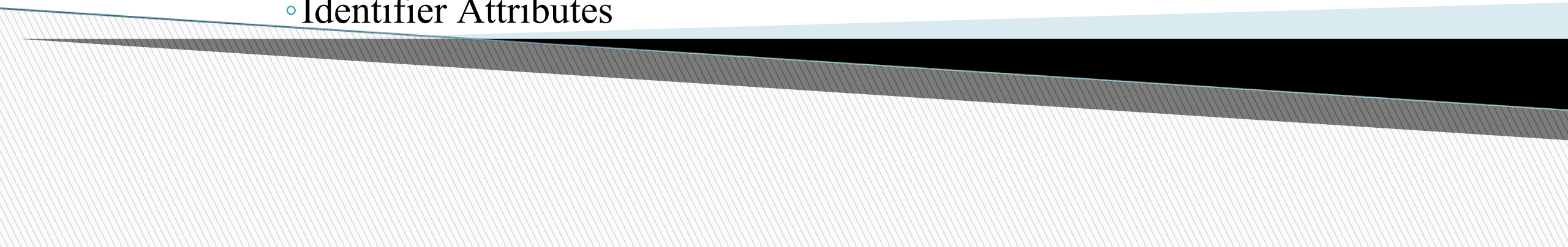
Attributes

Attributes

- ◆ Guidelines for naming attributes:
 - An attribute name is a noun
 - An attribute name should be unique
 - To make an attribute name unique and clear, each attribute name should follow a standard format
 - Similar attributes of different entity types should use similar but distinguishing names

Attributes

Classifications of attributes:

- Required versus Optional Attributes
 - Simple versus Composite Attribute
 - Single-Valued versus Multivalued Attribute
 - Stored versus Derived Attributes
 - Identifier Attributes
- 
- The bottom of the slide features a decorative graphic consisting of several overlapping, wavy horizontal bands. From top to bottom, the bands are light blue, black, dark grey, and light grey with a fine diagonal line pattern.

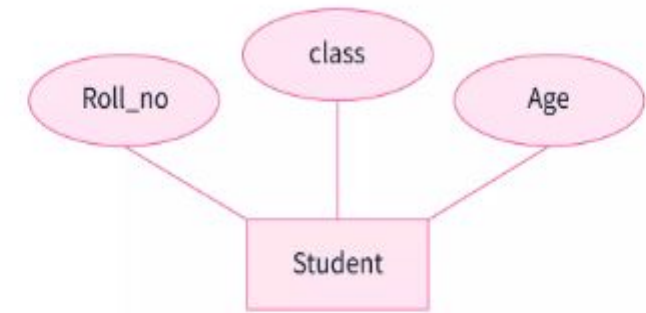
Simple Attributes

- ◆ Simple attributes in an ER model diagram are independent attributes that can't be classified further and also, can't be subdivided into any other component. These attributes are also known as **atomic attributes**.



Explanation

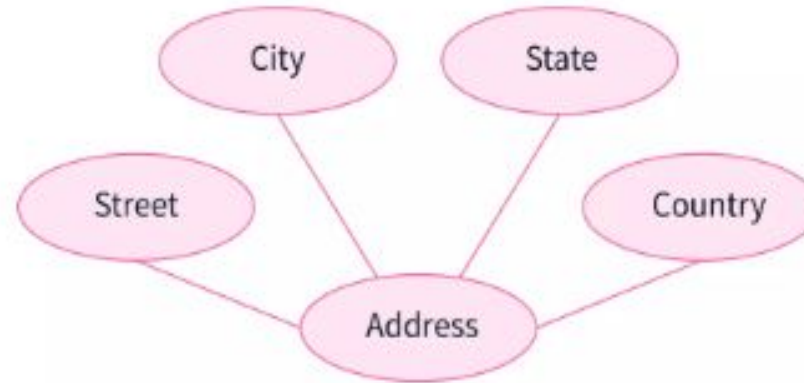
- ❖ Student is an entity represented by a rectangle, and it consists of attributes: Roll_no, class, and Age.
- ❖ Also, there is a point to be noted that we can't further subdivide the Roll_no attribute and even the other two attributes into sub-attributes.
- ❖ Hence, they are known as simple attributes of the Student entity.



Composite Attributes

- ❖ Composite attributes have opposite functionality to that of simple attributes as we can further subdivide composite attributes into different components or sub-parts that form simple attributes.
- ❖ **In simple terms, composite attributes are composed of one or more simple attributes.**

Example Diagram



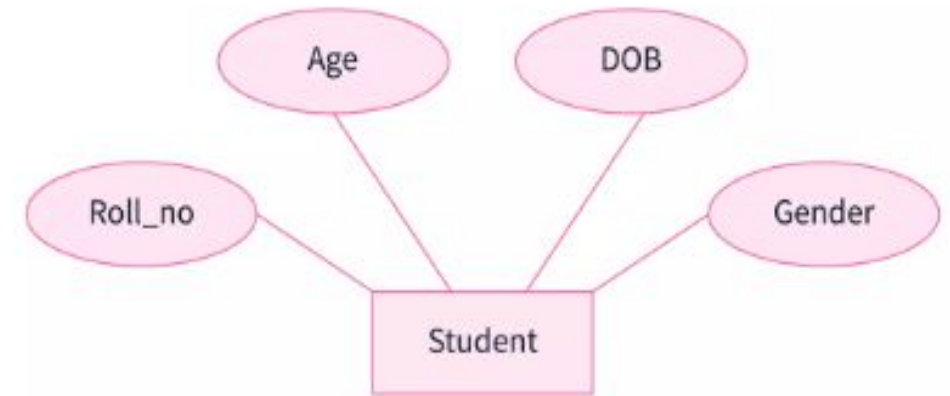
- ❖ As we can see in the above example, **Address** is a **composite attribute** represented by an elliptical shape, and it can be further subdivided into many simple attributes like Street, City, State, Country and Landmark etc.

Single-Valued Attributes

- ◆ Single valued attributes are those attributes that consist of a single value for each entity instance and can't store more than one value.
- ◆ The value of these single-valued attributes always remains the same, just like the name of a person.

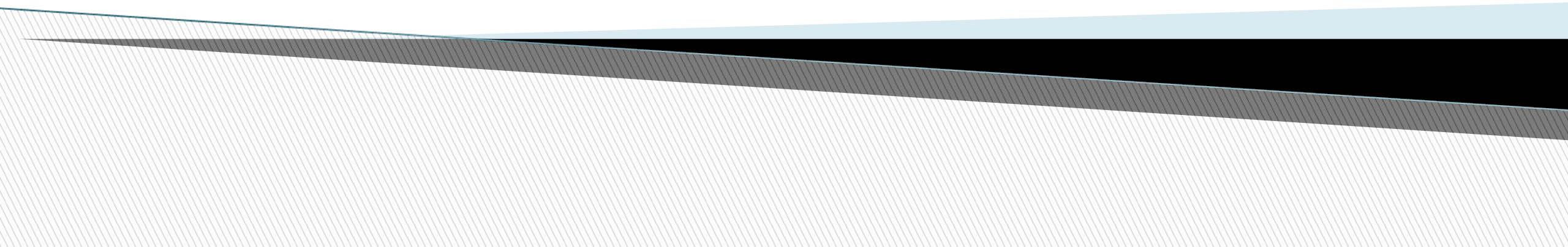
Explanation

- ❖ Student is an entity instance, and it consists of attributes: Roll_no, Age, DOB, and Gender.
- ❖ These attributes can store only one value from a set of possible values.
- ❖ Each entity instance can have only one Roll_no, which is a unique, single DOB by which we can calculate age and also fixed gender.
- ❖ Also, we can't further subdivide these attributes, and hence, they are



simple as well as single-valued attributes.

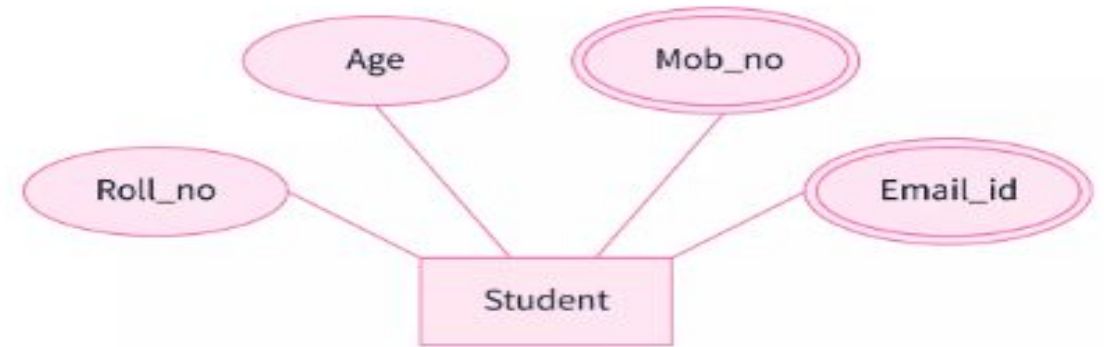
Multi-Valued Attributes

- ❖ Multi-valued attributes have opposite functionality to that of single-valued attributes, and as the name suggests, multi-valued attributes can take up and store more than one value at a time for an entity instance from a set of possible values.
 - ❖ These attributes are represented by **co-centric elliptical shape**, and we can also use **curly braces { }** to represent multi-valued attributes inside it.
- 

❖ The Student entity has four attributes: Roll_no and Age are simple as well as single-valued attributes as discussed but Mob_no and Email_id are represented by co-centric ellipse **are multi-valued attributes**.

❖ Each student in the real world can provide more than one email-id as well as a mobile contact number, and therefore, we need these attributes to be multi-valued so that they can store multiple values at a time for an entity instance.

:



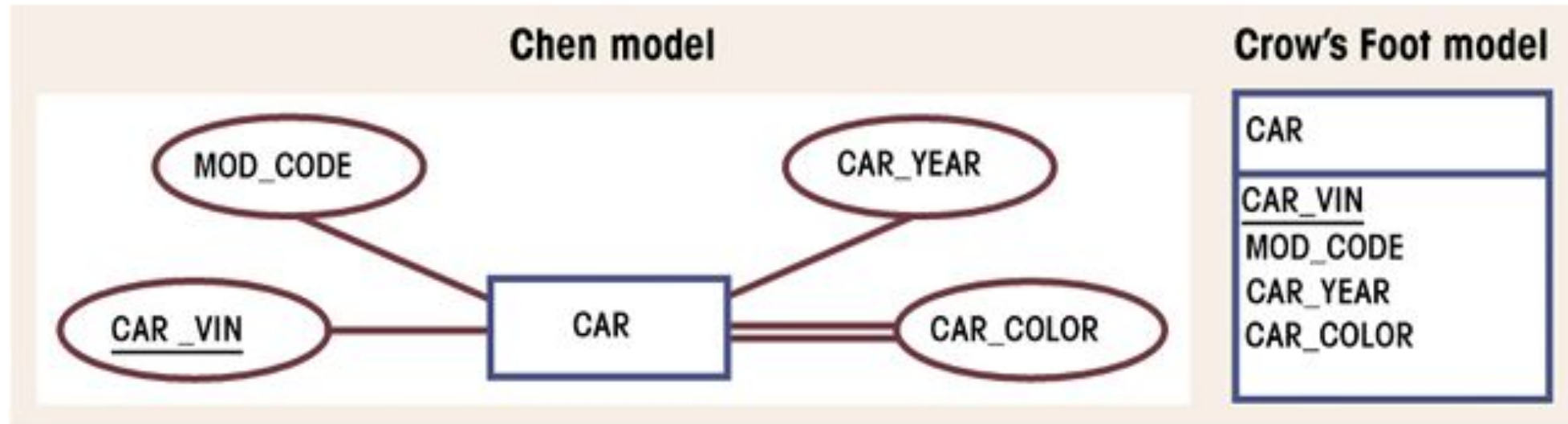


FIGURE 3.8 A MULTIVALUED ATTRIBUTE IN AN ENTITY

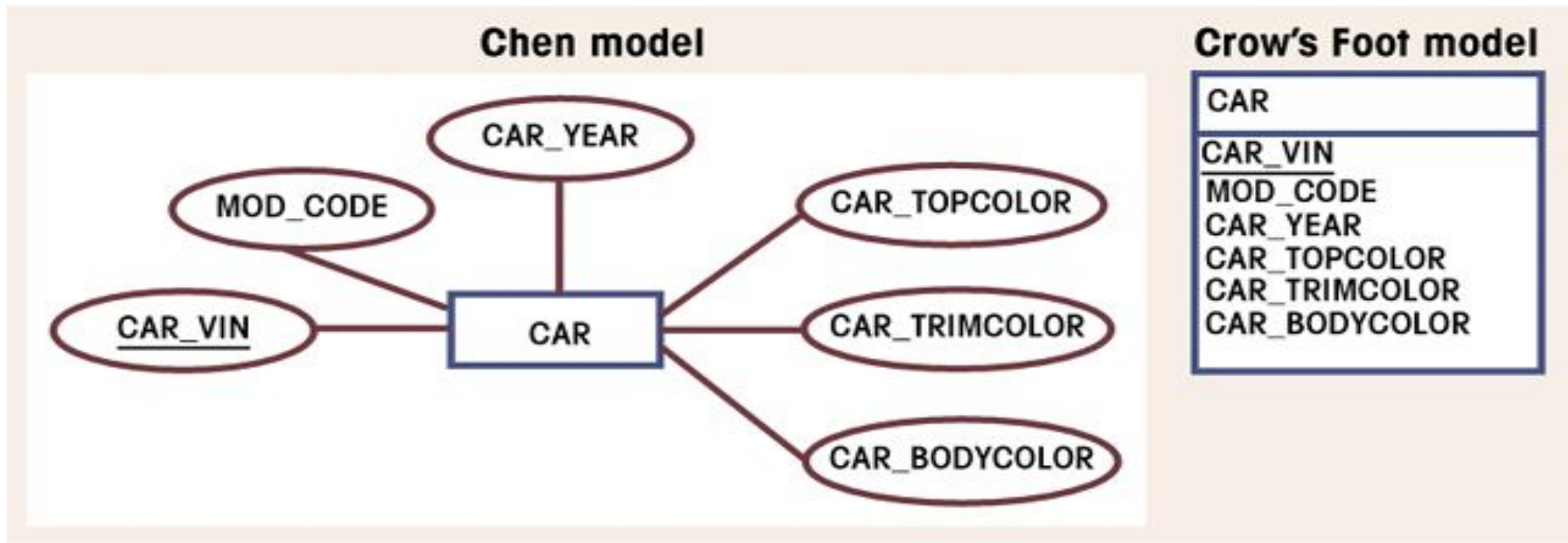
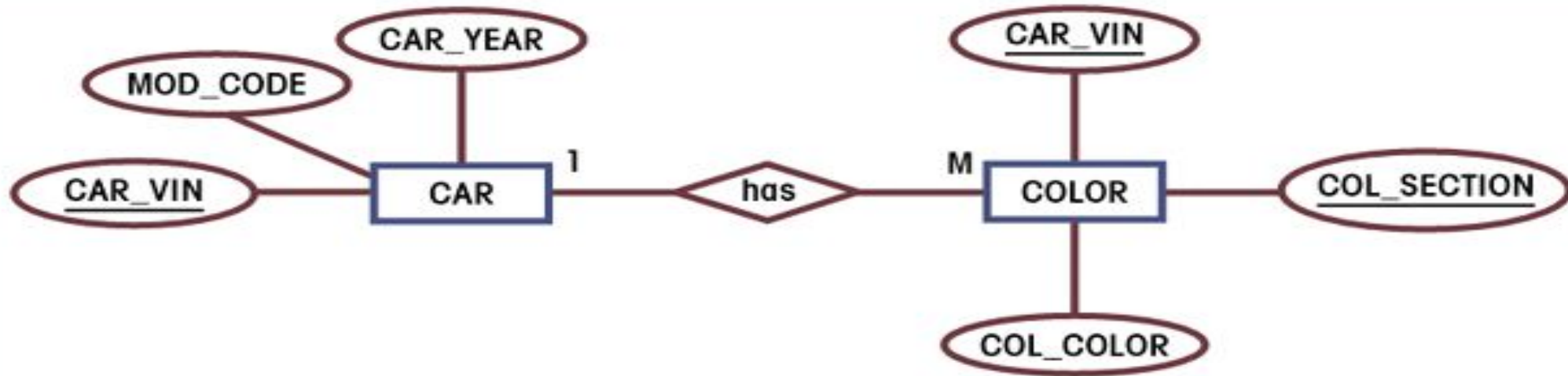


FIGURE 3.9 SPLITTING THE MULTIVALUED ATTRIBUTE INTO NEW ATTRIBUTES

Chen model



Crow's Foot model

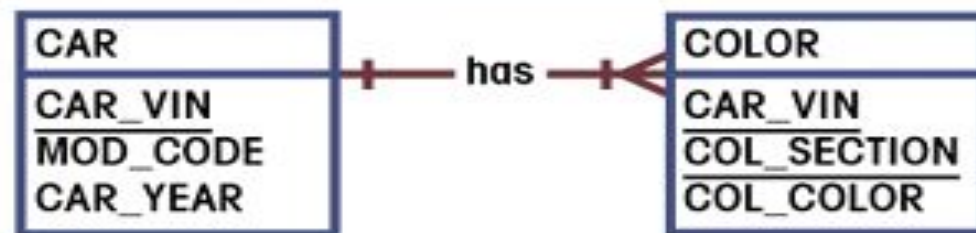


FIGURE 3.10 A NEW ENTITY SET COMPOSED OF A MULTIVALUED ATTRIBUTE'S COMPONENTS

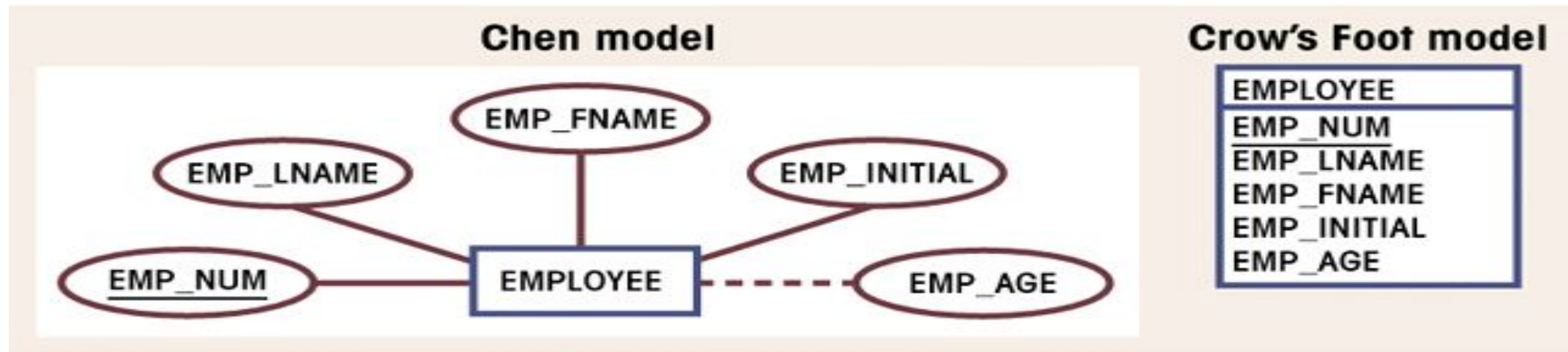
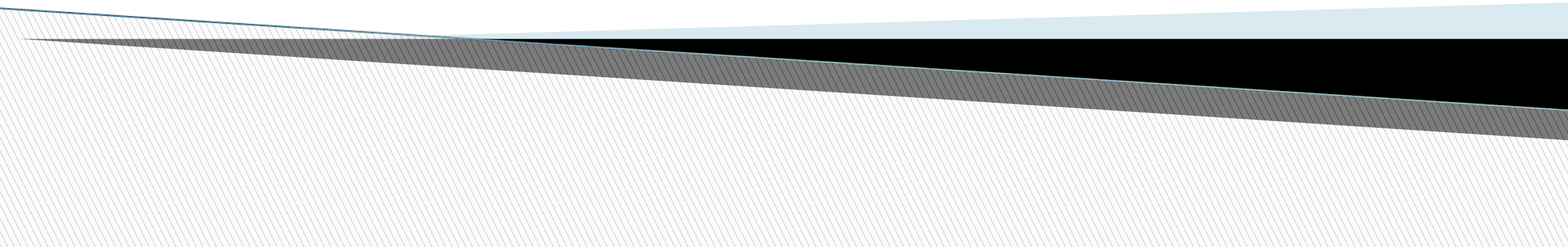
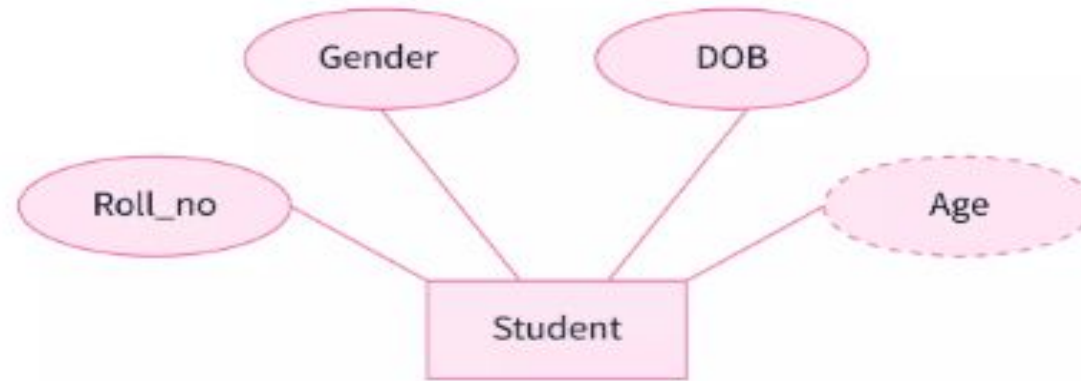


FIGURE 3.11 DEPICTION OF A DERIVED ATTRIBUTE

Derived Attributes

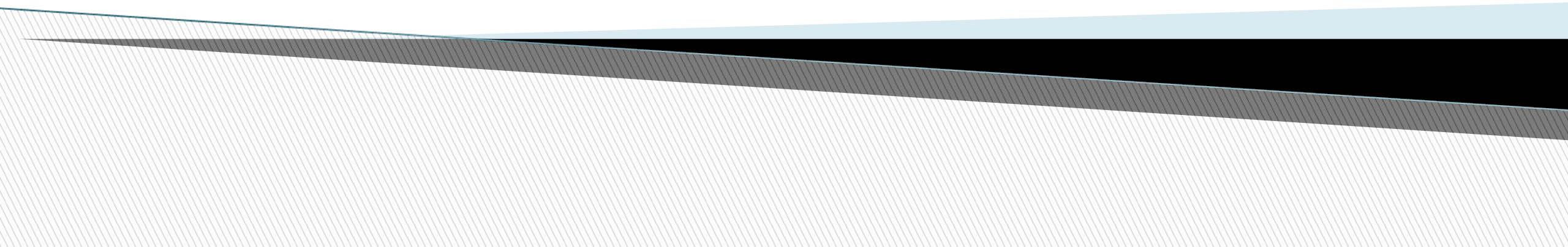
- ◆ Derived attributes in DBMS are those attributes whose values can be derived from the values of other attributes. They are always dependent upon other attributes for their value.
 - ◆ **For example:** DOB is a single-valued attribute and remains constant for an entity instance. From DOB, we can derive the Age attribute, which changes every year, and can easily calculate the age of a person from his/her date of birth value. Hence, the Age attribute here is derived attribute from the DOB single-valued attribute.
- 

Example Diagram:



- ❖ Derived attributes are always represented by dashed or dotted elliptical shapes.

Stored Attributes

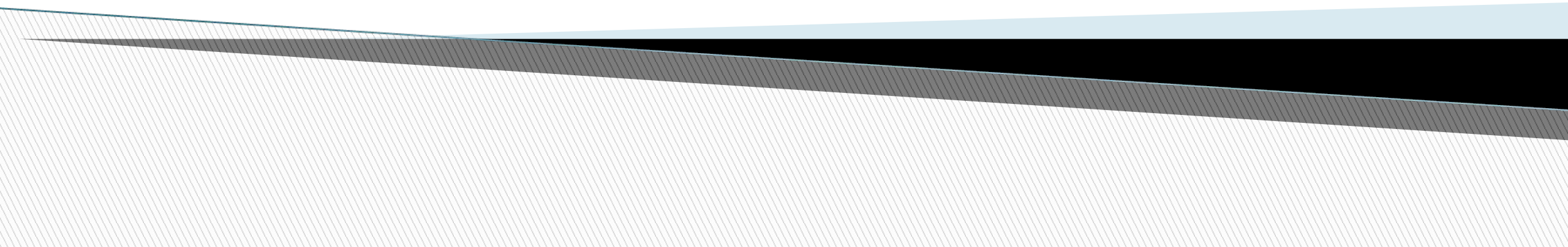
- ◆ Values of stored attributes remain constant and fixed for an entity instance and also, they help in deriving the derived attributes.
 - ◆ **For example**, the Age attribute can be derived from the Date of Birth attribute, and also, the Date of birth attribute has a fixed and constant value throughout the life of an entity. Hence, the Date of Birth attribute is a stored attribute.
- 

EXISTENCE DEPENDENCY

- ❖ An existence constraint or existence dependency can occur between two entities. The term existence dependency means that an instance of one entity cannot exist without the existence of an instance of the other (related) entity.

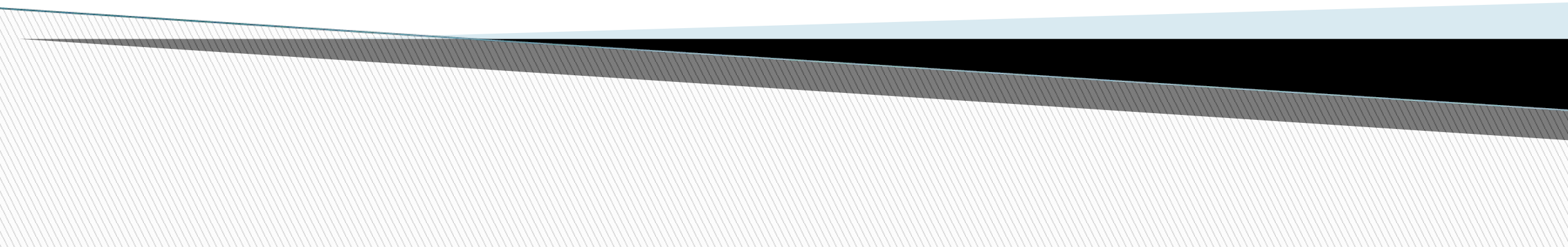
Weak Entity

An entity type that has an existence dependency is called **weak entity**.

- ❖ It can exist only if another entity exists. Thus, an instance of a weak entity cannot exist independently but depends on the existence of another entity (strong entity) instance.
 - ❖ The weak entity is also known as **child, or dependent or subordinate entity**. In E-R diagram, the existence dependency of weak entity is indicated by double line rectangle.
- 

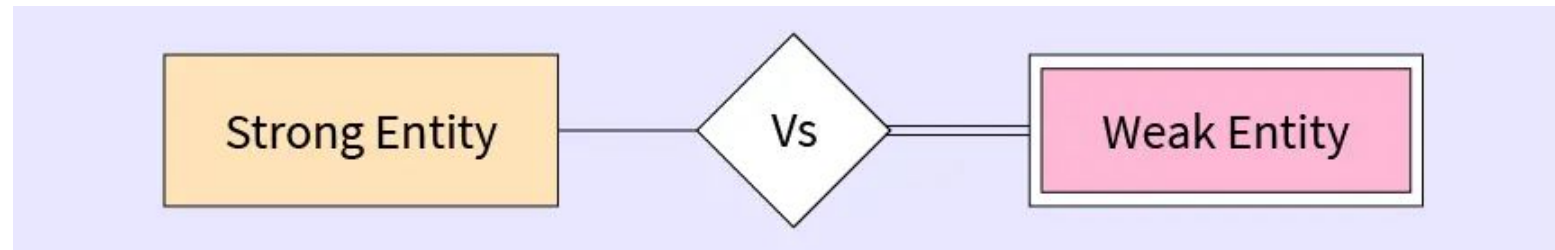
EXISTENCE DEPENDENCY

Strong Entity

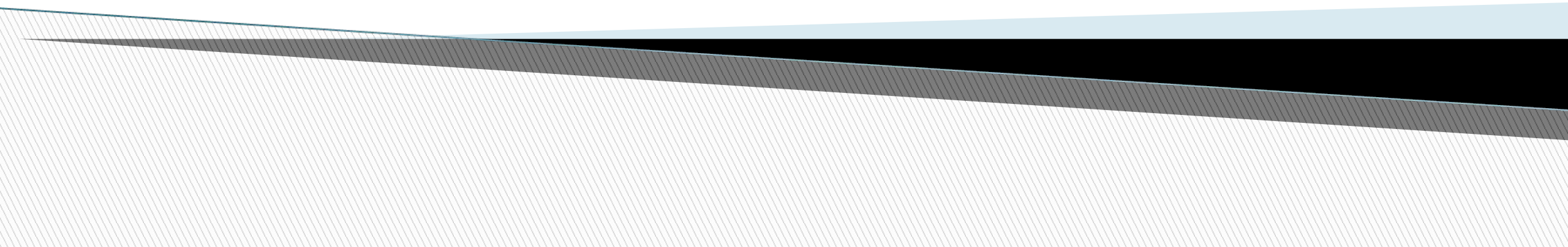
- ◆ An entity type that has no existence dependency is called **strong entity**.
 - ◆ It can exist without depending upon the existence of another entity.
 - ◆ The weak entity depends on the strong entity.
 - ◆ The strong entity is also known as **parent, or owner, or dominant entity**.
- 

Example

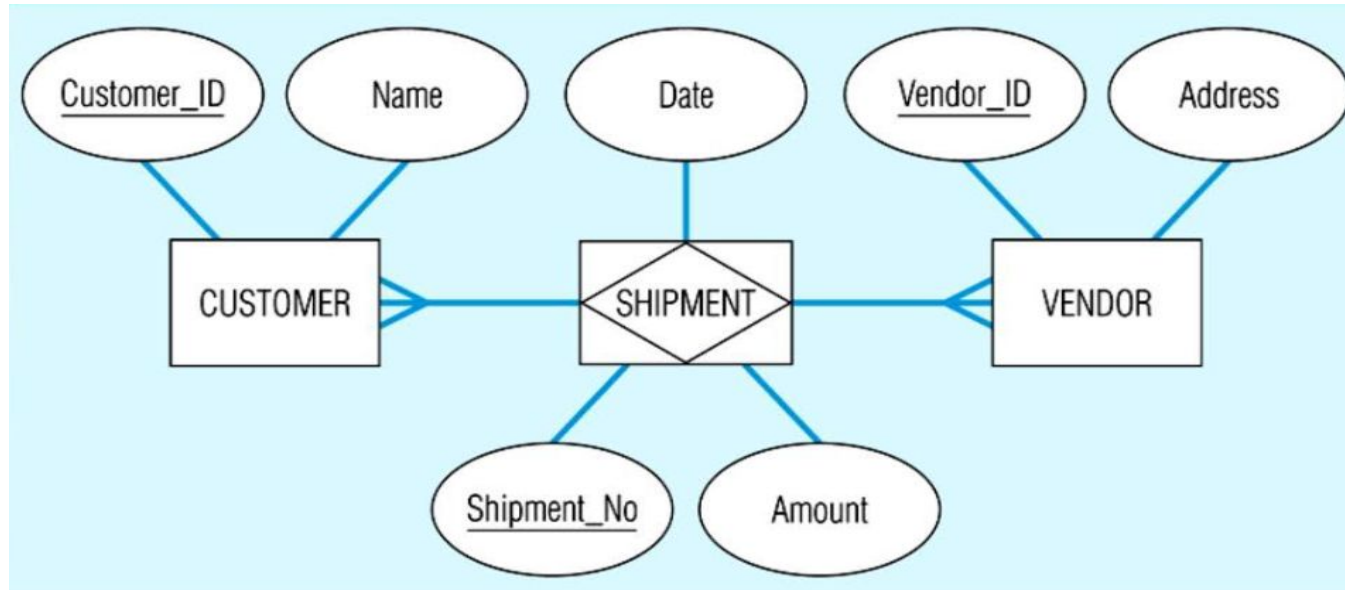
- ◆ In a college/university, a FACULTY entity is existence dependent on a DEPARTMENT entity. It means that a faculty member cannot exist without a corresponding DEPARTMENT.



Associative Entity

- ❖ Associative entities, additionally called relationship entities, represent relationships between two or more entities in a database.
 - ❖ They can also have attributes of their personal that describe homes of the relationship,
 - ❖ For Example, the begin date and give up date of an employee's participation in a venture.
- 

Associative Entity



Example:

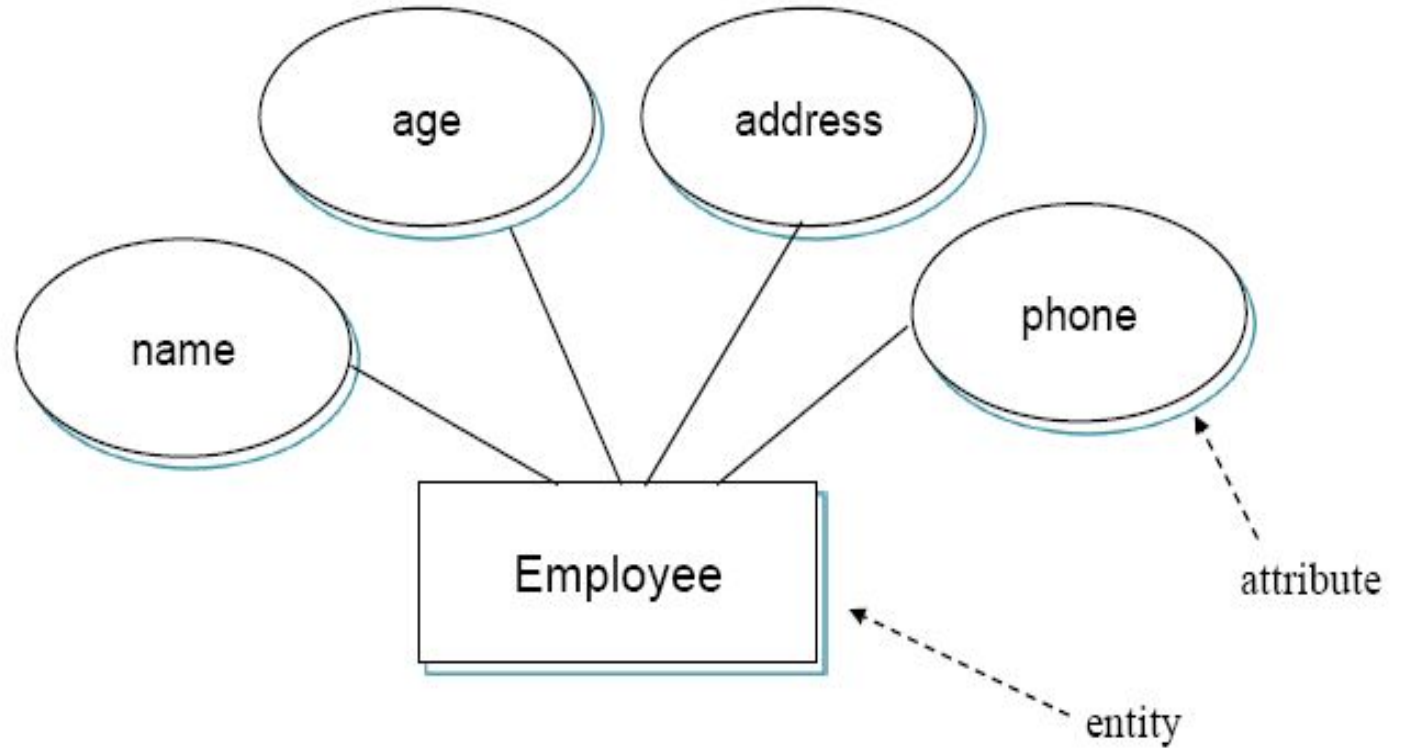
Taking the same example but adding more detail:

- A student can enroll in many courses.
- A course can have many students.
- But the **enrollment** might have its own details like `grade`, `date_enrolled`, etc.

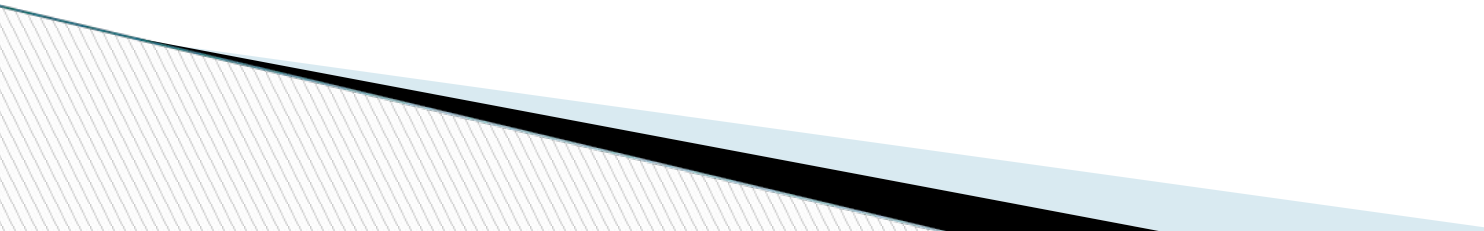
In this case, you'd make an **associative entity** called `Enrollment`, which includes:

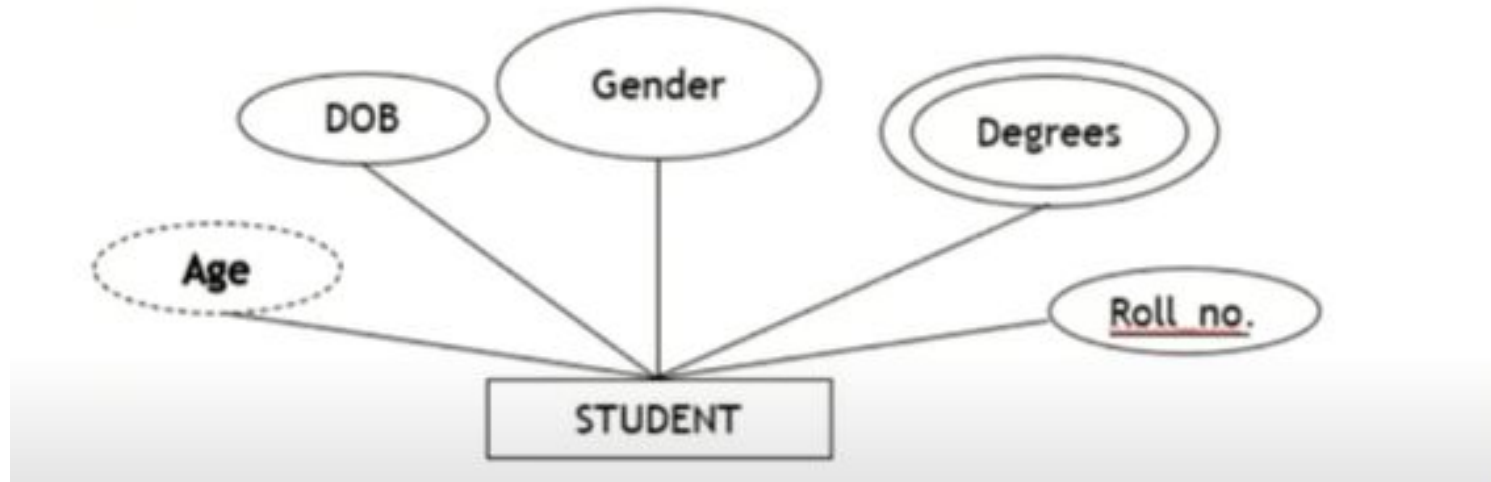
- Foreign keys to `Student` and `Course`
- Attributes like `grade`, `date_enrolled`

Example



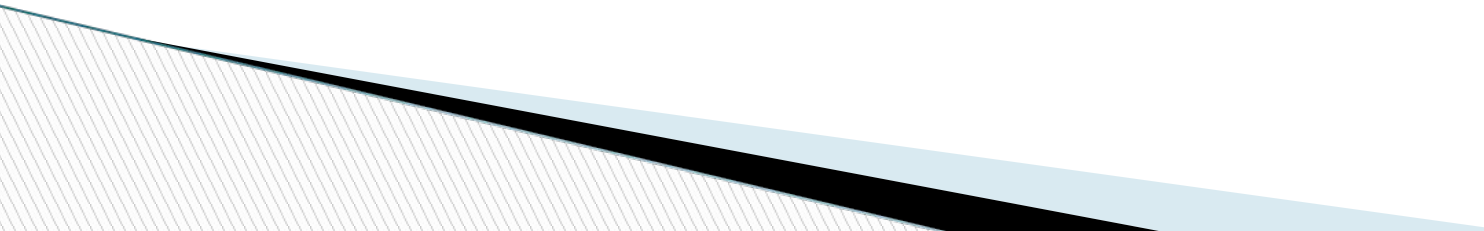
Identifiers

- Identifier is an attribute that identifies an entity instance among other instance in entity class.
 - For-example, there may be many instance of STUDENT entity class but each student is identified by his **roll_no**
 - Unique identifiers may identify only one entity instance.
 - A non-unique identifier may identify a set of instance.
- 



- Entity instances have **identifiers**
- An identifier will identify a particular instance in the entity class
 - SocialSecurityNumber
 - StudentID
 - EmployeeID

Characteristics of Identifiers

- Will not change in value.
 - Will not be null.
 - Substitute new, simple keys for long, composite keys.
- 

Identifier Types

□ Uniqueness

- Identifiers may be **unique** or **non-unique**
- If the identifier is unique, the data value for the identifier must be unique for all instances

□ Composite

- A **composite identifier** consists of 2 or more attributes. Whose values can be further divided.
 - E.g., Name can be divided in F_Name & L_Name

Con..

- ? We can also use more than two attributes for unique identification. Those attributes are called composite attributes. Here name and roll no can also be used combined as a composite identifier to identify a particular student.

